Sprint 06 Full Structured Notes: Refs, Effects, Cleanup, Custom Hooks, and Context API

0. Where You Are Before Sprint 06

Before Sprint 06, you should already understand these React ideas:

- React builds user interfaces using components.
- JSX lets you write UI-like syntax inside JavaScript.
- Components can receive data using props.
- The `children` prop lets one component wrap other JSX.
- `map()` helps render lists.
- Stable keys help React track list items.
- Events let React respond to clicks, typing, and form submissions.
- Controlled inputs store form values in React state.
- `event.preventDefault()` stops form reloads.
- `useState` lets React remember changing data.
- State updates should be immutable.
- Object state should be copied before changing a field.
- Array state should be updated with safe methods like spread, `filter()`, and `map()`.
- `useReducer` can organize related state actions.

Sprint 05 focused on this idea:

```
User action -> state changes -> React re-renders -> UI updates
```

Sprint 06 adds another important idea:

```
React renders UI -> then effects safely interact with the outside world
```

In simple words, Sprint 06 teaches you how React components can safely work with things outside normal rendering, such as the browser DOM, timers, document title, scroll events, reusable hook logic, and shared app-level state.

---

1. Sprint 06 Goal

The goal of Sprint 06 is to learn:

1. How to use `useRef` for DOM access and values that should not cause re-rendering.
2. How to use `useEffect` for side effects.

3. How to clean up timers, intervals, event listeners, and subscriptions.
4. How to create custom hooks for reusable logic.
5. How to use Context API for simple shared app state.

Simple version:

```
Sprint 06 teaches you how React safely works with the outside world.
```

---

2. Big Mental Model

React components should mostly be used to describe what the UI should look like.

Example:

```
return <h1>Hello React</h1>;
```

That is rendering.

But some tasks are not just rendering. For example:

- Focus an input.
- Change the browser tab title.
- Start a timer.
- Stop a timer.
- Listen for scroll.
- Remove a scroll listener.
- Debounce a search input.
- Share logged-in user data across components.

These tasks need special tools.

Sprint 06 tools:

Tool	Main job
`useRef`	Access DOM elements or store mutable values without re-rendering
`useEffect`	Run side-effect code after rendering
Cleanup function	Stop old timers, listeners, or subscriptions
Custom hook	Reuse hook-based logic
Context API	Share app-level values without prop drilling

Part One: `useRef`

3. What Is `useRef`?

`useRef` is a React Hook that gives you a ref object.

A ref object looks like this:

```
{
  current: something
}
```

The important part is `.current`.

Simple definition:

```
A ref is a box that can hold a value between renders without causing a re-
render.
```

Most beginner use case:

```
Use a ref to access a real DOM element, such as an input.
```

Example use cases:

- Focus an input.
- Read an uncontrolled input value.
- Measure an element.
- Store a value that should not trigger UI updates.

4. Basic `useRef` Syntax

Import `useRef` from React:

```
import { useRef } from "react";
```

Create a ref:

```
const inputRef = useRef(null);
```

Attach it to an element:

```
<input ref={inputRef} />
```

Use the DOM element through `.current`:

```
inputRef.current.focus();
```

---

5. `useRef` Mental Model

When you write:

```
const inputRef = useRef(null);
```

Think of it like this:

```
React creates a box.
At first, the box contains null.
After the input appears on the page, React puts the real input element into the
box.
You can access it with inputRef.current.
```

Timeline:

```
Before render:
inputRef.current = null

After input mounts:
inputRef.current = actual input element

After component unmounts:
inputRef.current = null again
```

---

6. Example: Focus an Input With `useRef`

```
import { useRef } from "react";

export default function SearchFocus() {
  const inputRef = useRef(null);

  function focusInput() {
    inputRef.current.focus();
  }

  return (
    <main>
      <h1>Search</h1>

      <input ref={inputRef} placeholder="Search..." />

      <button type="button" onClick={focusInput}>
        Focus input
      </button>
    </main>
  );
}
```

What happens:

```
User clicks button
-> focusInput runs
-> inputRef.current points to the input
-> .focus() focuses the input
```

---

7. Safer Ref Usage

Sometimes `.current` can be `null`.

So this can be unsafe:

```
inputRef.current.focus();
```

Safer version:

```
function focusInput() {
  if (inputRef.current) {
    inputRef.current.focus();
  }
}
```

Beginner rule:

```
Check ref.current before using it when there is any chance it could be null.
```

---

8. `useRef` vs `useState`

This is a very important distinction.

Use `useState` when changing the value should update the UI.

Use `useRef` when changing the value should not update the UI.

Situation	Use
Show count on screen and update it	`useState`
Store input text and validate it live	`useState`
Focus an input	`useRef`
Read DOM element directly	`useRef`
Store a timer ID	`useRef` or local variable inside effect
Store a value without re-rendering	`useRef`

Bad use of ref:

```
const countRef = useRef(0);
```

If the count should appear on screen, this is not the right tool.

Better:

```
const [count, setCount] = useState(0);
```

Simple rule:

```
If the screen should update, use state.
If you only need to remember or access something without updating the screen,
use ref.
```

---

9. Refs and Uncontrolled Inputs

An uncontrolled input is an input where the DOM stores the current value instead of React state.

Example:

```jsx
import { useRef } from "react";

export default function ReferralForm() {
  const refCodeInput = useRef(null);

  function handleSubmit(event) {
    event.preventDefault();

    const refCode = refCodeInput.current.value;
    console.log("Referral code:", refCode);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label htmlFor="refCode">Referral code</label>
      <input id="refCode" ref={refCodeInput} />

      <button type="submit">Submit</button>
    </form>
  );
}
```

Use uncontrolled inputs when:

- You only need the value on submit.
- You do not need live validation.
- You do not need live preview.
- The input value does not need to update the screen while typing.

For most beginner React forms, controlled inputs are still the better default.

Part Two: `useEffect`

---

10. What Is `useEffect`?

`useEffect` is a React Hook used for side effects.

A side effect is work that happens outside normal rendering.

Simple definition:

```
useEffect lets you run code after React has rendered the UI.
```

Examples of side effects:

- Changing `document.title`.
- Starting a timer.
- Starting an interval.
- Adding an event listener.
- Removing an event listener.
- Fetching data.
- Saving to `localStorage`.
- Connecting to a server.
- Subscribing to updates.

---

11. Why Effects Exist

React rendering should be mostly pure.

Pure rendering means:

```
Given the same props and state, the component returns the same UI.
```

Rendering should describe the UI.

Side effects should happen after rendering.

Bad mental model:

```
Render the component and directly do many outside-world operations inside the
render body.
```

Better mental model:

```
Render UI first.
Then useEffect runs outside-world work after render.
```

---

12. Basic `useEffect` Syntax

Import it:

```
import { useEffect } from "react";
```

Use it:

```
useEffect(() => {
  // side effect code here
}, [dependencies]);
```

Parts:

Part	Meaning
`useEffect`	React Hook for effects
`() => {}`	Function containing effect code
Dependency array	Controls when the effect runs

---

13. Example: Update `document.title`

```
import { useEffect, useState } from "react";

export default function TitleUpdater() {
  const [name, setName] = useState("");

  useEffect(() => {
    document.title = name ? `Hello, ${name}` : "React App";
```

```
  }, [name]);

  return (
    <main>
      <h1>Title Updater</h1>

      <label htmlFor="name">Name</label>
      <input
        id="name"
        value={name}
        onChange={(event) => setName(event.target.value)}
      />
    </main>
  );
}
```

What happens:

```
User types
-> name state changes
-> React re-renders
-> useEffect runs
-> document.title updates
```

Part Three: Dependency Arrays

---

14. What Is a Dependency Array?

The dependency array tells React when the effect should run.

It appears at the end of `useEffect`:

```
useEffect(() => {
  // effect code
}, [dependency]);
```

Simple definition:

```
The dependency array controls when useEffect runs again.
```

There are three main cases:

1. No dependency array.
2. Empty dependency array.
3. Dependency array with values.

---

15. Case 1: No Dependency Array

```
useEffect(() => {
  console.log("Effect ran");
});
```

Meaning:

```
Run after every render.
```

This can be useful sometimes, but beginners should be careful with it because it can run too often.

Example problem:

```
If state changes many times, this effect runs many times.
```

Beginner rule:

```
Do not omit the dependency array unless you really want the effect to run after
every render.
```

---

16. Case 2: Empty Dependency Array

```
useEffect(() => {
  console.log("Effect ran once");
}, []);
```

Meaning:

```
Run once after the component first appears.
```

Common uses:

- Load initial data.
- Set up an event listener once.
- Start something when the component mounts.

Important note:

```
[] does not mean “never run”.
It means “run once after the first render”.
```

---

17. Case 3: Dependency Array With Values

```
useEffect(() => {
  console.log("Query changed:", query);
}, [query]);
```

Meaning:

```
Run after the first render.
Run again whenever query changes.
```

This is usually the most useful pattern.

Example:

```
useEffect(() => {
  document.title = query ? `Search: ${query}` : "Search";
}, [query]);
```

Here, the effect depends on `query`.

So `query` should be in the dependency array.

---

18. Dependency Array Summary

Dependency style	When it runs
No array	After every render
`[]`	Once after first render
`[value]`	After first render and whenever `value` changes
`[a, b]`	After first render and whenever `a` or `b` changes

Memory rule:

```
If your effect uses a reactive value from the component, include it in the
dependency array.
```

Reactive values include:

- Props.
- State.
- Variables created inside the component.
- Functions created inside the component.

---

19. Common Dependency Mistake

Broken example:

```
useEffect(() => {
  console.log(query);
}, []);
```

Problem:

```
The effect uses query, but query is not in the dependency array.
```

Better:

```
useEffect(() => {
  console.log(query);
}, [query]);
```

Why?

Because when `query` changes, the effect should use the latest value.

Part Four: Cleanup Functions

---

20. What Is Cleanup?

A cleanup function is a function returned from `useEffect`.

It stops or removes something the effect started.

Basic pattern:

```
useEffect(() => {
  // start something

  return () => {
    // stop that thing
  };
}, []);
```

Simple definition:

```
Cleanup stops old side effects from staying alive.
```

---

21. Why Cleanup Is Important

Some effects start work that keeps running.

Examples:

- `setTimeout()`
- `setInterval()`
- Scroll listeners
- Resize listeners
- Subscriptions
- Server connections

If you do not clean them up, you can get bugs like:

- Duplicate timers.
- Duplicate event listeners.
- Memory leaks.
- Old values being used.
- Console logs repeating too many times.
- Code running after a component is gone.

Simple mental model:

```
Effect starts something.
Cleanup stops it.
```

---

22. Cleanup With `setTimeout`

```
useEffect(() => {
  const id = setTimeout(() => {
    console.log("Timer finished");
  }, 1000);

  return () => {
    clearTimeout(id);
  };
}, []);
```

What happens:

```
Effect starts a timeout.
Cleanup cancels the timeout if needed.
```

---

23. Cleanup With `setInterval`

```
useEffect(() => {
  const id = setInterval(() => {
    console.log("Interval running");
  }, 1000);

  return () => {
```

```
    clearInterval(id);
  };
}, []);
```

Why cleanup is needed:

```
setInterval keeps running until you stop it.
clearInterval stops it.
```

---

24. Cleanup With Event Listeners

Example: scroll listener.

```
import { useEffect, useState } from "react";

export default function ScrollMessage() {
  const [scrolled, setScrolled] = useState(false);

  useEffect(() => {
    function handleScroll() {
      setScrolled(true);
    }

    window.addEventListener("scroll", handleScroll);

    return () => {
      window.removeEventListener("scroll", handleScroll);
    };
  }, []);

  return (
    <main style={{ minHeight: "150vh" }}>
      <h1>Scroll Listener</h1>
      {scrolled && <p>You scrolled.</p>}
    </main>
  );
}
```

What happens:

```
Component appears
-> effect adds scroll listener
```

```
-> user scrolls
-> handleScroll runs
-> component unmounts
-> cleanup removes scroll listener
```

Beginner rule:

```
If you add an event listener in an effect, remove it in cleanup.
```

---

25. Effect Lifecycle Mental Model

Effects can run more than once.

When dependencies change, React usually does this:

```
1. Cleanup old effect
2. Run new effect
```

When component unmounts, React does this:

```
Cleanup final effect
```

Visual model:

```
Render
-> effect starts timer/listener

Dependency changes
-> cleanup old timer/listener
-> run new effect

Component unmounts
-> cleanup final timer/listener
```

Part Five: Debouncing and Delayed Preview

---

26. What Is Debouncing?

Debouncing means waiting until the user stops doing something before running logic.

Common example:

```
User types in search box.
Do not search after every single key press.
Wait until the user pauses typing.
Then run the search.
```

Simple definition:

```
Debouncing delays an update until changes stop for a short time.
```

Why it is useful:

- Avoids too many API calls.
- Avoids too many expensive calculations.
- Makes search inputs smoother.
- Prevents unnecessary work while the user is still typing.

---

27. Example: Delayed Preview

```jsx
import { useEffect, useState } from "react";

export default function DelayedPreview() {
  const [text, setText] = useState("");
  const [preview, setPreview] = useState("");

  useEffect(() => {
    const id = setTimeout(() => {
      setPreview(text);
    }, 700);

    return () => {
      clearTimeout(id);
    };
  }, [text]);

  return (
    <main>
```

```
      <h1>Delayed Preview</h1>

      <label htmlFor="message">Message</label>
      <input
        id="message"
        value={text}
        onChange={(event) => setText(event.target.value)}
      />

      <p>Instant text: {text}</p>
      <p>Delayed preview: {preview}</p>
    </main>
  );
}
```

What this teaches:

```
User types
-> text updates immediately
-> effect starts a timeout
-> user types again before 700ms
-> cleanup cancels old timeout
-> new timeout starts
-> preview updates only after user stops typing for 700ms
```

This is the basic idea behind debouncing.

Part Six: Custom Hooks

---

28. What Is a Custom Hook?

A custom hook is a reusable function that uses React Hooks inside it.

Simple definition:

```
A custom hook lets you reuse hook logic across components.
```

Custom hooks must start with `use`.

Examples:

- `useDebounce`
- `useFetch`
- `useLocalStorage`
- `useAuth`
- `useWindowSize`

---

29. Why Custom Hooks Must Start With `use`

React has rules for hooks.

Hooks should be called:

- At the top level.
- Inside React components.
- Inside custom hooks.

The `use` prefix helps React tools and linting tools understand:

```
This function is a hook and must follow Hook rules.
```

Correct custom hook name:

```
useDebounce
```

Bad custom hook name:

```
debounceValue
```

The second name may work as a normal function name, but it does not clearly tell React tooling that this function uses hooks.

---

30. What Logic Should Become a Custom Hook?

Move logic into a custom hook when:

- You repeat the same hook logic in multiple components.
- The component is becoming too crowded.
- The logic is not mainly about displaying JSX.

- The logic has a clear reusable purpose.

Examples:

Logic	Possible custom hook
Debounced value	`useDebounce`
Fetching data	`useFetch`
Reading/writing localStorage	`useLocalStorage`
Auth context access	`useAuth`
Window size tracking	`useWindowSize`

---

31. Build a `useDebounce` Custom Hook

Create this file:

```
src/hooks/useDebounce.js
```

Code:

```
import { useEffect, useState } from "react";

export function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => {
      clearTimeout(id);
    };
  }, [value, delay]);

  return debouncedValue;
}
```

What this hook receives:

Parameter	Meaning
`value`	The value that changes quickly
`delay`	How long to wait before updating

What this hook returns:

```
The delayed version of the value.
```

---

32. How `useDebounce` Works

```
const [debouncedValue, setDebouncedValue] = useState(value);
```

This stores the delayed value.

```
useEffect(() => {
  const id = setTimeout(() => {
    setDebouncedValue(value);
  }, delay);

  return () => {
    clearTimeout(id);
  };
}, [value, delay]);
```

This means:

```
When value or delay changes:
1. Start a timer.
2. After delay milliseconds, update debouncedValue.
3. If value changes before the timer finishes, cleanup cancels the old timer.
```

```
return debouncedValue;
```

This gives the delayed value back to the component.

---

33. Use `useDebounce` in a SearchBox

```
import { useEffect, useRef, useState } from "react";
import { useDebounce } from "./hooks/useDebounce";

export default function SearchBox() {
  const inputRef = useRef(null);
  const [query, setQuery] = useState("");

  const debouncedQuery = useDebounce(query, 500);

  useEffect(() => {
    document.title = debouncedQuery
      ? `Search: ${debouncedQuery}`
      : "Search";
  }, [debouncedQuery]);

  function focusInput() {
    if (inputRef.current) {
      inputRef.current.focus();
    }
  }

  return (
    <main>
      <h1>Search Box</h1>

      <label htmlFor="search">Search</label>
      <input
        id="search"
        ref={inputRef}
        value={query}
        onChange={(event) => setQuery(event.target.value)}
        placeholder="Type to search..."
      />

      <button type="button" onClick={focusInput}>
        Focus input
      </button>

      <p>Instant query: {query}</p>
      <p>Debounced query: {debouncedQuery}</p>
    </main>
  );
}
```

Concepts combined in this example:

- `useState` for controlled input.
- `useRef` for input focusing.
- `useDebounce` for delayed value.
- `useEffect` for updating `document.title`.
- Cleanup inside the custom hook.
- Accessible label connected to input.
- Real button with `type="button"`.

Part Seven: Context API

---

34. What Problem Does Context Solve?

Props are useful because they pass data from parent to child.

But sometimes many components need the same shared data.

Example:

```
App
 └── Layout
      └── Header
           └── Navbar
                └── UserMenu
```

Suppose only `UserMenu` needs the logged-in user.

Without Context, you may need to pass `user` like this:

```
App -> Layout -> Header -> Navbar -> UserMenu
```

This is called prop drilling.

Simple definition:

```
Prop drilling means passing props through components that do not actually need
them, just to reach a deeper child.
```

---

35. What Is Context API?

Context API is a built-in React feature for sharing values with many components.

Simple definition:

```
Context lets components read shared data without passing props through every
level.
```

Good beginner use cases:

- Current logged-in user.
- Theme mode.
- Language.
- Cart summary.

Do not use Context for every state value.

Bad idea:

```
Putting every input field into Context.
```

Better:

```
Keep local state local.
Use Context only when many parts of the app need the same value.
```

---

36. Context API Has Three Main Pieces

Context has three basic pieces:

1. `createContext()`
2. Provider
3. `useContext()`

---

37. Piece 1: `createContext()`

`createContext()` creates a context object.

```jsx
import { createContext } from "react";

const AuthContext = createContext(null);
```

Simple idea:

```
createContext creates the shared data container.
```

The `null` is the default value.

---

38. Piece 2: Provider

The Provider makes a value available to components below it.

```jsx
<AuthContext.Provider value={value}>
  {children}
</AuthContext.Provider>
```

Simple idea:

```
Provider shares the value with everything inside it.
```

If a component is outside the Provider, it cannot read that provided value.

---

39. Piece 3: `useContext()`

`useContext()` reads the context value.

```jsx
import { useContext } from "react";

const auth = useContext(AuthContext);
```

Simple idea:

```
useContext lets a child component read the shared value.
```

---

40. Full Auth Context Example

Create this file:

```
src/context/AuthContext.jsx
```

Code:

```jsx
import { createContext, useContext, useState } from "react";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  function login(name) {
    setUser({ name });
  }

  function logout() {
    setUser(null);
  }

  const value = {
    user,
    login,
    logout,
  };

  return (
    <AuthContext.Provider value={value}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  return useContext(AuthContext);
}
```

What this file does:

Code	Meaning
`createContext(null)`	Creates the context
`AuthProvider`	Wraps part of the app and provides auth data
`user`	Stores the logged-in user
`login(name)`	Sets a user object
`logout()`	Clears the user
`value`	Object shared through Context
`useAuth()`	Custom hook for reading auth context

---

41. Why Create a `useAuth` Hook?

Instead of writing this in every component:

```
const auth = useContext(AuthContext);
```

You can write:

```
const auth = useAuth();
```

This is cleaner.

It also hides the direct context details from normal components.

Simple rule:

```
A custom context hook like useAuth makes context easier to use.
```

---

42. Navbar Using Auth Context

Create this file:

```
src/components/Navbar.jsx
```

Code:

```jsx
import { useAuth } from "../context/AuthContext";

export function Navbar() {
  const { user, login, logout } = useAuth();

  return (
    <nav>
      {user ? (
        <>
          <p>Logged in as {user.name}</p>
          <button type="button" onClick={logout}>
            Logout
          </button>
        </>
      ) : (
        <button type="button" onClick={() => login("Sangeeth")}>
          Login
        </button>
      )}
    </nav>
  );
}
```

What happens:

```
If user exists:
Show logged-in message and Logout button.

If user is null:
Show Login button.
```

---

43. Use the Provider in `App.jsx`

```jsx
import { useState } from "react";
import { AuthProvider } from "./context/AuthContext";
import { Navbar } from "./components/Navbar";

export default function App() {
  const [localNote, setLocalNote] = useState("");

  return (
    <AuthProvider>
```

```
      <main>
        <Navbar />

        <h1>Sprint 06 Practice</h1>

        <label htmlFor="note">Local note</label>
        <input
          id="note"
          value={localNote}
          onChange={(event) => setLocalNote(event.target.value)}
        />

        <p>This local note does not need Context: {localNote}</p>
      </main>
    </AuthProvider>
  );
}
```

Important point:

```
localNote stays local because only App needs it.
user goes in Context because Navbar and other app sections may need it.
```

This prevents overusing Context.

---

44. Context Tree Mental Model

```
App
 └── AuthProvider
      ├── Navbar -> reads user from context
      └── Dashboard -> can also read user from context
```

Provider makes the value available below it.

Components below the Provider can read the value using `useContext()` or a custom hook like `useAuth()`.

---

45. When to Use Context

Use Context when:

- Several components need the same value.
- Passing props through many levels becomes messy.
- The value is app-level or section-level state.

Good examples:

- Current user.
- Theme.
- Language.
- Cart count.

Avoid Context when:

- Only one component needs the value.
- The state changes extremely often and is shared widely without care.
- You are using it just to avoid normal props between parent and child.

Simple rule:

```
Use props for normal parent-child data.
Use Context for shared app-level data.
Keep local state local.
```

Part Eight: Debug Task

---

46. Broken Code

```
useEffect(() => {
  const id = setInterval(() => console.log(query), 1000);
}, []);

function focusInput() {
  inputRef.current.focus();
}
```

This code has three main problems.

---

47. Problem 1: Missing Cleanup

This starts an interval:

```
setInterval(() => console.log(query), 1000);
```

But it never stops the interval.

Problem:

```
The interval keeps running.
This can cause duplicate logs or memory leaks.
```

Fix:

```
return () => {
  clearInterval(id);
};
```

---

48. Problem 2: Missing Dependency

The effect uses `query`:

```
console.log(query)
```

But the dependency array is empty:

```
[]
```

Problem:

```
The effect may use an old query value.
```

Fix:

```
[query]
```

49. Problem 3: Unsafe Ref Access

This can break if `inputRef.current` is `null`:

```
inputRef.current.focus();
```

Safer:

```
if (inputRef.current) {
  inputRef.current.focus();
}
```

---

50. Fixed Debug Code

```
useEffect(() => {
  const id = setInterval(() => {
    console.log(query);
  }, 1000);

  return () => {
    clearInterval(id);
  };
}, [query]);

function focusInput() {
  if (inputRef.current) {
    inputRef.current.focus();
  }
}
```

What is fixed:

- Interval is cleaned up.
- `query` is included as a dependency.
- Ref is checked before use.

Part Nine: Common Sprint 06 Mistakes

---

51. Mistake 1: Using Ref Instead of State for UI Data

Wrong:

```
const countRef = useRef(0);
```

If the count should update the screen, use state:

```
const [count, setCount] = useState(0);
```

Rule:

```
Screen updates need state, not refs.
```

---

52. Mistake 2: Not Checking `ref.current`

Risky:

```
inputRef.current.focus();
```

Safer:

```
if (inputRef.current) {
  inputRef.current.focus();
}
```

---

53. Mistake 3: Forgetting the Dependency Array

Risky:

```
useEffect(() => {
  document.title = query;
});
```

This runs after every render.

Better:

```
useEffect(() => {
  document.title = query;
}, [query]);
```

---

54. Mistake 4: Using `[]` When the Effect Depends on State

Wrong:

```
useEffect(() => {
  console.log(query);
}, []);
```

Better:

```
useEffect(() => {
  console.log(query);
}, [query]);
```

---

55. Mistake 5: Missing Cleanup for Timers

Wrong:

```
useEffect(() => {
  setTimeout(() => {
    console.log("Done");
  }, 1000);
}, []);
```

Better:

```
useEffect(() => {
  const id = setTimeout(() => {
    console.log("Done");
  }, 1000);

  return () => {
```

```
    clearTimeout(id);
  };
}, []);
```

---

56. Mistake 6: Missing Cleanup for Event Listeners

Wrong:

```
useEffect(() => {
  window.addEventListener("scroll", handleScroll);
}, []);
```

Better:

```
useEffect(() => {
  window.addEventListener("scroll", handleScroll);

  return () => {
    window.removeEventListener("scroll", handleScroll);
  };
}, []);
```

---

57. Mistake 7: Naming a Custom Hook Without `use`

Wrong:

```
function debounce(value, delay) {
  // uses hooks
}
```

Better:

```
function useDebounce(value, delay) {
  // uses hooks
}
```

---

58. Mistake 8: Overusing Context

Wrong idea:

```
Put every small state value into Context.
```

Better idea:

```
Keep local state local.
Use Context only for shared app-level values.
```

Part Ten: Practice Projects

---

59. Practice Project 1: Ref Focus App

Goal:

```
Build an input and a button. Clicking the button focuses the input.
```

Requirements:

- Use `useRef(null)`.
- Attach the ref to an input.
- Use a real button.
- Check `inputRef.current` before calling `.focus()`.

Starter:

```jsx
import { useRef } from "react";

export default function RefFocusApp() {
  const inputRef = useRef(null);

  function handleFocus() {
    if (inputRef.current) {
      inputRef.current.focus();
    }
  }
```

```
  return (
    <main>
      <h1>Ref Focus App</h1>

      <label htmlFor="search">Search</label>
      <input id="search" ref={inputRef} />

      <button type="button" onClick={handleFocus}>
        Focus input
      </button>
    </main>
  );
}
```

---

60. Practice Project 2: Document Title App

Goal:

```
Create a controlled input. When the user types, update document.title.
```

Requirements:

- Use `useState`.
- Use `useEffect`.
- Include the state value in the dependency array.
- Use a connected label.

Starter:

```
import { useEffect, useState } from "react";

export default function DocumentTitleApp() {
  const [title, setTitle] = useState("");

  useEffect(() => {
    document.title = title || "React App";
  }, [title]);

  return (
    <main>
      <h1>Document Title App</h1>

      <label htmlFor="title">Page title</label>
```

```
      <input
        id="title"
        value={title}
        onChange={(event) => setTitle(event.target.value)}
      />
    </main>
  );
}
```

---

61. Practice Project 3: Delayed Preview App

Goal:

```
Type text. Show a preview after 700ms.
```

Requirements:

- Use controlled input.
- Use `useEffect`.
- Use `setTimeout`.
- Clean up with `clearTimeout`.
- Include the correct dependency.

Starter:

```
import { useEffect, useState } from "react";

export default function DelayedPreviewApp() {
  const [text, setText] = useState("");
  const [preview, setPreview] = useState("");

  useEffect(() => {
    const id = setTimeout(() => {
      setPreview(text);
    }, 700);

    return () => {
      clearTimeout(id);
    };
  }, [text]);

  return (
    <main>
```

```
      <h1>Delayed Preview</h1>

      <label htmlFor="text">Text</label>
      <input
        id="text"
        value={text}
        onChange={(event) => setText(event.target.value)}
      />

      <p>Instant: {text}</p>
      <p>Delayed: {preview}</p>
    </main>
  );
}
```

---

62. Practice Project 4: SearchBox With Custom Hook

Goal:

```
Extract debounce logic into useDebounce and use it in a SearchBox.
```

Files:

```
src/hooks/useDebounce.js
src/App.jsx
```

`src/hooks/useDebounce.js`:

```
import { useEffect, useState } from "react";

export function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => {
      clearTimeout(id);
    };
  }, [value, delay]);
```

```
  return debouncedValue;
}
```

`src/App.jsx`:

```
import { useEffect, useRef, useState } from "react";
import { useDebounce } from "./hooks/useDebounce";

export default function App() {
  const inputRef = useRef(null);
  const [query, setQuery] = useState("");
  const debouncedQuery = useDebounce(query, 500);

  useEffect(() => {
    document.title = debouncedQuery
      ? `Search: ${debouncedQuery}`
      : "Search";
  }, [debouncedQuery]);

  function handleFocus() {
    if (inputRef.current) {
      inputRef.current.focus();
    }
  }

  return (
    <main>
      <h1>Search Box</h1>

      <label htmlFor="search">Search</label>
      <input
        id="search"
        ref={inputRef}
        value={query}
        onChange={(event) => setQuery(event.target.value)}
      />

      <button type="button" onClick={handleFocus}>
        Focus search
      </button>

      <p>Instant query: {query}</p>
      <p>Debounced query: {debouncedQuery}</p>
    </main>
```

```
  );
}
```

---

63. Practice Project 5: Auth Context App

Goal:

```
Build a simple AuthProvider with user, login, and logout.
```

Files:

```
src/context/AuthContext.jsx
src/components/Navbar.jsx
src/App.jsx
```

`src/context/AuthContext.jsx`:

```jsx
import { createContext, useContext, useState } from "react";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  function login(name) {
    setUser({ name });
  }

  function logout() {
    setUser(null);
  }

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
```

```
  return useContext(AuthContext);
}
```

`src/components/Navbar.jsx`:

```
import { useAuth } from "../context/AuthContext";

export function Navbar() {
  const { user, login, logout } = useAuth();

  return (
    <nav>
      {user ? (
        <>
          <p>Logged in as {user.name}</p>
          <button type="button" onClick={logout}>
            Logout
          </button>
        </>
      ) : (
        <button type="button" onClick={() => login("Sangeeth")}>
          Login
        </button>
      )}
    </nav>
  );
}
```

`src/App.jsx`:

```
import { useState } from "react";
import { AuthProvider } from "./context/AuthContext";
import { Navbar } from "./components/Navbar";

export default function App() {
  const [localNote, setLocalNote] = useState("");

  return (
    <AuthProvider>
      <main>
        <Navbar />

        <h1>Auth Context Practice</h1>

        <label htmlFor="note">Local note</label>
```

```
        <input
          id="note"
          value={localNote}
          onChange={(event) => setLocalNote(event.target.value)}
        />

        <p>Local note: {localNote}</p>
      </main>
    </AuthProvider>
  );
}
```

Important lesson:

```
user is shared through Context.
localNote stays local in App.
```

Part Eleven: Revision Questions and Answers

---

64. Question 1: When should you use a ref instead of state?

Use a ref when you need to access or remember something without causing a re-render.

Examples:

- Focus an input.
- Read a DOM element.
- Store a mutable value that does not affect the UI.

Use state when the value should update the screen.

---

65. Question 2: What does `useRef(null)` mean?

It creates a ref object whose `.current` value starts as `null`.

Example:

```
const inputRef = useRef(null);
```

After the input is rendered with `ref={inputRef}`, React puts the real input element into:

```
inputRef.current
```

---

66. Question 3: What is `useEffect` used for?

`useEffect` is used for side effects.

Examples:

- Updating `document.title`.
- Starting timers.
- Adding event listeners.
- Fetching data.
- Saving to localStorage.

Simple answer:

```
useEffect runs code after React renders.
```

---

67. Question 4: What does an empty dependency array mean?

```
useEffect(() => {
  console.log("Run once");
}, []);
```

It means:

```
Run once after the first render.
```

---

68. Question 5: What does no dependency array mean?

```
useEffect(() => {
  console.log("Run after every render");
});
```

It means:

```
Run after every render.
```

---

69. Question 6: What does `[query]` mean in a dependency array?

```
useEffect(() => {
  console.log(query);
}, [query]);
```

It means:

```
Run after the first render and whenever query changes.
```

---

70. Question 7: Why do we need cleanup?

Cleanup stops old side effects from continuing after they are no longer needed.

Examples:

- Stop old timers.
- Stop intervals.
- Remove event listeners.
- Close subscriptions or connections.

Without cleanup, apps can have memory leaks and duplicate behavior.

---

71. Question 8: How do you clean up a timeout?

```
useEffect(() => {
  const id = setTimeout(() => {
    console.log("Done");
  }, 1000);

  return () => {
    clearTimeout(id);
```

```
  };
}, []);
```

---

72. Question 9: How do you clean up an event listener?

```
useEffect(() => {
  function handleScroll() {
    console.log("Scrolled");
  }

  window.addEventListener("scroll", handleScroll);

  return () => {
    window.removeEventListener("scroll", handleScroll);
  };
}, []);
```

---

73. Question 10: What is a custom hook?

A custom hook is a reusable function that uses React Hooks inside it.

Example:

```
function useDebounce(value, delay) {
  // hook logic
}
```

It must start with `use`.

---

74. Question 11: What is debouncing?

Debouncing means delaying work until changes stop for a short time.

Example:

```
Do not update search results after every key press.
Wait until the user stops typing.
```

---

75. Question 12: What problem does Context solve?

Context helps avoid prop drilling.

It lets components read shared values without passing props through every level.

Example shared values:

- Logged-in user.
- Theme.
- Language.

---

76. Question 13: What are the three basic Context API pieces?

The three basic pieces are:

1. `createContext()`
2. Provider
3. `useContext()`

---

77. Question 14: Is Context a replacement for all state?

No.

Context is not for every small state value.

Use local state when only one component needs the value.

Use Context when many components need the same shared value.

Part Twelve: Sprint 06 Mini Revision Sheet

---

78. `useRef`

```
const inputRef = useRef(null);
```

Used for:

- DOM access.
- Mutable values that do not trigger re-rendering.

Attach to element:

```
<input ref={inputRef} />
```

Use safely:

```
if (inputRef.current) {
  inputRef.current.focus();
}
```

---

79. `useEffect`

```
useEffect(() => {
  // side effect
}, [dependency]);
```

Used for:

- Effects after render.
- Browser APIs.
- Timers.
- Event listeners.
- External systems.

---

80. Dependency Array

```
useEffect(() => {
  // runs after every render
});

useEffect(() => {
  // runs once after first render
}, []);

useEffect(() => {
```

```
  // runs when value changes
}, [value]);
```

---

81. Cleanup

```
useEffect(() => {
  const id = setTimeout(() => {
    console.log("Done");
  }, 1000);

  return () => {
    clearTimeout(id);
  };
}, []);
```

Memory rule:

```
If an effect starts something, ask whether cleanup should stop it.
```

---

82. Custom Hook

```
function useSomething() {
  // can use React hooks here
}
```

Rules:

- Name starts with `use`.
- Can use built-in hooks.
- Reuses hook logic.
- Does not directly render JSX by default.

---

83. `useDebounce`

```
export function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(value);
```

```
  useEffect(() => {
    const id = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => {
      clearTimeout(id);
    };
  }, [value, delay]);

  return debouncedValue;
}
```

Meaning:

```
Return a delayed version of value.
```

---

84. Context API

Create context:

```
const AuthContext = createContext(null);
```

Provide value:

```
<AuthContext.Provider value={{ user, login, logout }}>
  {children}
</AuthContext.Provider>
```

Read value:

```
const { user } = useContext(AuthContext);
```

Cleaner custom hook:

```
export function useAuth() {
  return useContext(AuthContext);
}
```

Part Thirteen: Definition of Done

---

85. Sprint 06 Is Complete When You Can Do These

You are ready to move on from Sprint 06 when you can:

- Explain what `useRef` does.
- Focus an input with `useRef`.
- Explain why refs do not replace state.
- Explain what `useEffect` does.
- Use `useEffect` to update `document.title`.
- Explain the three dependency array patterns.
- Add cleanup for `setTimeout`.
- Add cleanup for `setInterval`.
- Add and remove an event listener correctly.
- Build a delayed preview with cleanup.
- Extract debounce logic into `useDebounce`.
- Use a custom hook inside a component.
- Explain prop drilling.
- Create context with `createContext()`.
- Provide context with a Provider.
- Read context with `useContext()`.
- Build a simple `AuthProvider` with `user`, `login`, and `logout`.
- Keep unrelated local state outside Context.

---

86. Final Sprint 06 Summary

Sprint 06 teaches you how React components safely interact with things outside normal rendering.

Main tools:

```
useRef -> access DOM or store values without re-rendering
useEffect -> run side effects after render
cleanup -> stop old timers, listeners, and subscriptions
custom hooks -> reuse hook logic
Context API -> share app-level values without prop drilling
```

The most important beginner rules are:

```
Use state when the screen should update.
Use refs when you need access without re-rendering.
```

```
Use effects for outside-world work.
Clean up anything that keeps running.
Use custom hooks to reuse logic.
Use Context for shared app-level data, not every local state value.
```

You are ready for Sprint 07 when you can build these without copying:

1. A ref-focused input.
2. A document title updater.
3. A delayed preview with cleanup.
4. A `useDebounce` custom hook.
5. A debounced search box.
6. A simple Auth Context with login/logout.